

Twoja pierwsza aplikacja z agentem

Powtórzysz metodę z Zeszytu 2 w miniaturze — od koncepcji po działającą aplikację — w jedno popołudnie. Z gotowymi promptami do skopiowania.

Zeszyt 4 z 4 · Pakiet „Jak powstała aplikacja tic-bot-toe”

Wymaga ukończonego Zeszytu 3 (działający Claude Code).

Stan na 18.07.2026

Czego się tu nauczysz

Zbudujesz **własną, działającą aplikację** — prostą listę zadań, która zapisuje dane w przeglądarce. Nie chodzi o listę zadań samą w sobie. Chodzi o to, żebyś **przeszedł tę samą pętlę**, którą przeszedł tic-bot-toe: koncepcja → mini-specyfikacja → plan etapowy → realizacja → weryfikacja.

Każdy krok ma **prompt do skopiowania** i komentarz **dlaczego** jest tak sformułowany. Kopiuj prompty do panelu Claude Code w VS Code.

WSKAZOWKA

Dlaczego lista zadań? Jedna technologia (strona w przeglądarce), zero serwera, zero baz danych, a mimo to prawdziwa aplikacja: dodaje, usuwa, zapamiętuje. Wykonalna w jedno popołudnie. Gdy ją skończysz, ten sam schemat przeniesiesz na własny pomysł.

Zanim zaczniesz

1. Otwórz VS Code i utwórz nowy, pusty folder, np. `moja-lista-zadan` (**File** → **Open Folder**).
2. Otwórz panel Claude Code i upewnij się, że jest zalogowany (Zeszyt 3, krok 7).
3. Czytaj po kolei. Nie przeskakuj kroków — metoda polega na kolejności.

WSKAZOWKA

Czym jest tu Claude Code. W tym pakiecie używamy Claude Code jako **rozszerzenia do VS Code** (zainstalowanego w Zeszytcie 3) — agent mieszka w bocznym panelu edytora. To nie jedyna droga; ta sama metoda (koncepcja → plan → etapy → weryfikacja) działa też w innych narzędziach:

- **Claude Code jako CLI** — to samo narzędzie uruchamiane w terminalu poleceniem `claude` w folderze projektu (instalacja: `npm install -g @anthropic-ai/claude-code`). Wygodne, gdy wolisz pracę w wierszu poleceń zamiast panelu.
- **GitHub Copilot** — inny agent kodujący wbudowany w VS Code (osobne konto/subskrypcja GitHub). Interfejs się różni, ale prompty z tego zeszytu i cała metoda pozostają takie same.

Wybierz jedno. Prompty poniżej wklejasz do panelu/okna swojego narzędzia — treść jest identyczna niezależnie od wyboru.

Krok 1 – Koncepcja (rola „Cowork”)

W tic-bot-toe koncepcja powstała w rozmowie, zanim padła pierwsza linia kodu. Zrobisz to samo — tu w miniaturze. Możesz użyć zwykłego czatu na claude.ai (tryb myślenia) albo od razu Claude Code. Poproś o **mini-specyfikację**, nie o kod.

SKOPIUJ DO AGENTA

Jestem początkujący. Pomóż mi zaprojektować **bardzo prostą aplikację: listę zadań**, która działa jako pojedyncza strona internetowa i **zapisuje dane w przeglądarce** (bez serwera i bez bazy danych).

Napisz **mini-specyfikację na jedną stronę**: krótki cel, listę funkcji (dodawanie zadania, oznaczanie jako zrobione, usuwanie, zapamiętywanie po odświeżeniu), świadome NIE-cele (czego NA RAZIE nie robimy) oraz proponowaną technologię dla początkującego. Nie pisz jeszcze kodu — najpierw ustalmy zakres.

Dlaczego tak: prosisz o **specyfikację, nie o kod** — dokładnie jak w projekcie tic-bot-toe, gdzie „SPEC jest źródłem prawdy”. Prosisz też o **NIE-cele**, bo połowa sukcesu to wiedzieć, czego *nie* budujemy na start.

OK - PUNKT KONTROLNY

Masz przed sobą krótki dokument: cel, ~4 funkcje, kilka NIE-celów i propozycję technologii (najpewniej: jeden plik HTML z JavaScriptem i zapisem w „localStorage”). Przeczytaj i, jeśli trzeba, poproś o korektę zakresu.

Krok 2 – Plan etapowy z „definicją ukończenia”

Duże rzeczy robi się kawałkami. Poproś o plan — i o **kryterium**, po którym poznasz, że etap jest skończony.

SKOPIUJ DO AGENTA

Na podstawie tej mini-specyfikacji rozpisz **plan w 3–4 etapach**. Każdy etap ma mieć: (1) co dokładnie powstaje, (2) **definicję ukończenia** — co muszę zobaczyć/kliknąć, żeby uznać etap za zrobiony. Ułóż etapy tak, żeby po **każdym** aplikacja się uruchamiała i coś już działało. Nie zaczynaj jeszcze kodować.

Dlaczego tak: „definicja ukończenia” to ta sama zasada, która trzymała w ryzach tic-bot-toe — „nie przechodź dalej z czerwonymi testami”. Chcesz, żeby po każdym etapie było **coś działającego**, a nie dziesięć niedokończonych kawałków naraz.

OK - PUNKT KONTROLNY

Masz plan 3–4 etapów, a przy każdym jasne kryterium „skończone, gdy...”. Np. Etap 1: pusta strona z polem i przyciskiem; Etap 2: dodawanie zadań na listę; Etap 3: oznaczanie/usuwanie; Etap 4: zapamiętywanie po odświeżeniu.

Krok 3 – Realizacja, etap po etapie

Teraz budujemy. Reguła jest jedna: **jeden etap naraz**, sprawdzasz, dopiero potem kolejny.

SKOPIUJ DO AGENTA

Zrealizuj **tylko Etap 1** z planu. Pracuj w tym folderze. Trzymaj się mini-specyfikacji. Gdy skończysz, napisz krótko: co powstało, jaki plik otworzyć i jak sprawdzić, że Etap 1 spełnia swoją definicję ukończenia. **Nie zaczynaj Etapu 2.**

Dlaczego tak: „tylko Etap 1” i „nie zaczynaj Etapu 2” — świadomie ograniczasz agenta, żeby móc **sprawdzić** efekt, zanim ruszy dalej. To Ty trzymasz tempo.

Po sprawdzeniu (patrz Krok 4) przechodzisz dalej:

SKOPIUJ DO AGENTA

Etap 1 działa. Zrealizuj teraz **tylko Etap 2**. Po skończeniu powiedz, jak sprawdzić jego definicję ukończenia. Nie ruszaj kolejnych etapów.

Powtarzaj ten wzorec aż do ostatniego etapu. Za każdym razem: **jeden etap → sprawdź → następny**.

UWAGA

Jeśli agent chce zrobić „wszystko naraz”, przypomnij mu: „proszę tylko bieżący etap”. Budowanie warstwami to najważniejsza ochrona przed chaosem — dokładnie ta lekcja, którą odrobił tic-bot-toe.

Krok 4 – Weryfikacja (na własne oczy)

Testy i zapewnienia agenta to nie wszystko — w tic-bot-toe błąd wyszedł dopiero przy sprawdzeniu **na żywo**. Rób tak samo.

1. Otwórz aplikację. Dla pojedynczego pliku HTML: kliknij go dwukrotnie w folderze albo poproś agenta:

SKOPIUJ DO AGENTA

Jak najprościej uruchomić tę aplikację na moim komputerze, żeby zobaczyć ją w przeglądarce? Podaj krok po kroku.

1. **Przeklikaj** to, co miało działać: dodaj zadanie, oznacz jako zrobione, usuń, **odśwież stronę** i sprawdź, czy dane zostały.
2. Jeśli coś nie działa — opisz to konkretnie i poproś o poprawkę (wzorzec „code-review”):

SKOPIUJ DO AGENTA

Sprawdziłem na żywo. Działa: [co działa]. Nie działa: [dokładnie co zrobiłem i co się stało zamiast tego]. Znajdź przyczynę i popraw, potem powiedz, jak zweryfikować, że jest już dobrze.

Dlaczego tak: podajesz **konkretny objaw** („kliknąłem X, stało się Y, spodziewałem się Z”), a nie „nie działa”. Im konkretniej, tym celniejsza poprawka — tak samo jak z dobrym briefem z Zeszytu 1.

OK - PUNKT KONTROLNY

Przeszedłeś całą listę funkcji z mini-specyfikacji, klikając na żywo, i wszystko działa — łącznie z **zapamiętywaniem po odświeżeniu**. Masz **działającą, własną aplikację**.

Krok 5 — Zapisz postęp (git) i rozbuduj

Zapisz stan w git

Skoro to działa, zapisz „punkt kontrolny”, do którego zawsze wrócisz.

SKOPIUJ DO AGENTA

Zainicjuj repozytorium git w tym folderze, dodaj sensowny plik `.gitignore` i zrób pierwszy commit z opisową wiadomością. Wyjaśnij mi w dwóch zdaniach, co właśnie zrobiłeś.

Dlaczego tak: od tej chwili każdy kolejny eksperyment jest bezpieczny — zawsze cofniesz się do działającej wersji. To ta sama higiena, co 71 commitów w tic-bot-toe.

Rozbuduj (opcjonalnie)

Masz fundament — teraz dokładaj funkcje **po jednej**, tym samym wzorcem (nowy mini-etap → realizacja → weryfikacja). Pomysły:

- filtry „wszystkie / aktywne / zrobione”,
- edycja treści zadania po kliknięciu,
- licznik pozostałych zadań,

- ładniejszy wygląd (poproś o konkretny styl, np. „ciemny, minimalistyczny”).

SKOPIUJ DO AGENTA

Chcę dołożyć **jedną** funkcję: [opisz ją w jednym zdaniu]. Zaproponuj krótką definicję ukończenia, zrealizuj i powiedz, jak sprawdzić efekt. Nie zmieniaj niczego poza tą funkcją.

Pułapki początkującego (i jak ich uniknąć)

Pułapka	Jak jej uniknąć
„Zrób wszystko naraz”	Trzymaj się jednego etapu; wprost proś „tylko bieżący etap”.
„Nie działa” bez szczegółów	Opisuj objaw: co kliknąłeś, co się stało, czego oczekiwałeś.
Ślepe zaufanie	Zawsze sprawdzaj na żywo, nie tylko na słowo agenta.
Rozrost zakresu	Wróć do mini-specyfikacji i NIE-celów; nowe pomysły = osobny mini-etap.
Strach przed zepsuciem	Commituj po każdym działającym etapie — cofnięcie jest zawsze możliwe.

Podsumowanie – masz już metodę

Właśnie przeszedłeś dokładnie tę samą pętlę, co projekt z tego pakietu, tylko w mniejszej skali:

1. **Koncepcja** → mini-specyfikacja (rola „Cowork”).
2. **Plan** etapowy z definicją ukończenia.
3. **Realizacja** etap po etapie w Claude Code.
4. **Weryfikacja** na żywo + poprawki (wzorzec code-review).
5. **Zapis** w git i rozbudowa po jednej funkcji.

To jest przenośne. Następnym razem podstaw pod „listę zadań” **swój** pomysł — a te same pięć kroków doprowadzi Cię do celu. Tak samo, jak z 30-minutowej rozmowy w samochodzie powstała arena grająca na żywo pod ticbottoe.lol.

OK - PUNKT KONTROLNY

Twoja pierwsza aplikacja działa i jest zapisana w git. Potrafisz nazwać pięć kroków metody i wiesz, że kolejny projekt zaczniesz **od koncepcji i planu**, nie od kodu. Gratulacje — to jest cały sekret.